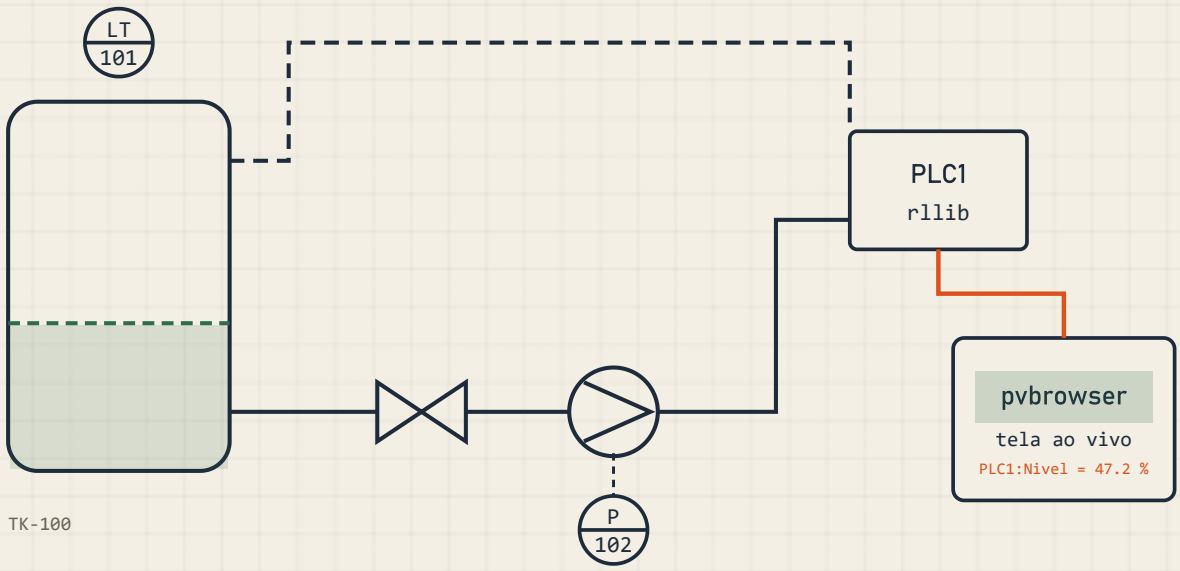
SCADA / IHM ABERTO

PV Suite

SOFTWARE LIVRE
FEITO PARA A PLANTA

Telas de supervisão montadas no Designer, ligadas ao CLP sem rodeios, rodando em Windows, Linux e macOS.



do sensor à tela, sem escrever código

VERSÃO
1.0.0-beta.4

PLATAFORMAS
Windows · Linux · macOS

LICENÇA
GPL / LGPL

01 · A IDEIA

O que é o PV Suite

O PV Suite parte do pvbrowser, um SCADA livre com muitos anos de chão de fábrica, leva-o para o Qt 6 e acrescenta o que mais tempo tomava à mão: os tags, o daemon de comunicação e a lógica de cada botão.

A proposta é simples: uma tela de supervisão feita com o mouse, o vínculo com o CLP declarado num só lugar e o código próprio ainda disponível quando for preciso algo mais fino.

Não substitui quem programa; tira das mãos dele o que é repetitivo. O que antes eram centenas de linhas para ler um registro, escalá-lo e pintá-lo de uma cor agora é uma linha numa tabela.

PARA QUEM É

- Integradores que montam painéis e supervisão para clientes pequenos e médios.
- Técnicos de manutenção que querem ver a planta sem depender de licenças caras.
- Professores e alunos de automação que precisam de algo real para praticar.
- Quem já tem projetos em pvbrowser e quer levá-los ao Qt 6 sem refazê-los.

> *«A tela deve mostrar o que acontece na planta, não o quanto custou programá-la.»*

As quatro peças

PB

PVBROWSER

O cliente: mostra as telas que o servidor envia. Inclui navegador web.

PS

PVSERVER

A biblioteca com que se programam os servidores de telas, um por projeto.

PD

PVDEVELOP

O ambiente de trabalho: editor, Designer de telas, tags e gerador de daemons.

RL

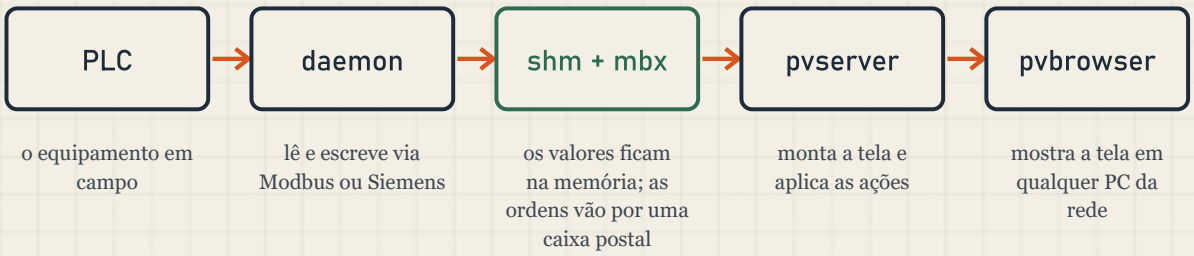
RLLIB

A comunicação: Modbus, Siemens, memória compartilhada, caixas postais e tags.

02 · POR DENTRO

Como um dado viaja

Cada peça faz uma coisa só e a faz bem. Assim uma falha de comunicação não derruba as telas, e uma tela travada não para a leitura do CLP.



O daemon é gerado pelo pvdevelop a partir da tabela de tags. Lê o CLP em ciclos, deixa cada valor com sua qualidade na memória compartilhada e atende as escritas que chegam pela caixa postal.

O pvserver não fala com o CLP: lê essa memória. Se o daemon parar, a tela percebe e marca o dado como ruim em vez de mostrar um número velho como se fosse novo.

O pvbrowser é leve: recebe a tela pela rede e devolve os cliques. Vários podem ficar abertos ao mesmo tempo, em PCs diferentes.

SEM ESCREVER CÓDIGO

Nas Propriedades de cada widget, o botão **Tags...** define o que ele faz e como aparece. O pvdevelop gera o código ao salvar a tela. Passando o mouse sobre cada campo, uma ajuda diz para que serve e em quais widgets se usa.

AÇÕES

escrever um valor, alternar, somar ou subtrair com limites, digitar um número, ir para outra tela, pedir confirmação, agir só se uma condição for cumprida.

ANIMAÇÕES

texto com formato, cor por regras, piscar, visibilidade, imagem conforme o valor, sequências de imagens e marca de dado ruim.

```
//pvb: btnBomba press=write:PLC1:Bomba=1
  release=write:PLC1:Bomba=0
color=PLC1:Bomba
```

O que traz

TAGS COM NOME

Estruturas, vetores e bits de inteiros (`PLC1:Estado.3`). Definidos uma vez, usados em todas as telas.

SIMULAÇÃO

O daemon inicia com `-sim` contra um simulador Modbus: tudo se testa sem o CLP ao lado.

MONITOR DE TAGS

Valores ao vivo, com qualidade. Permite escrever e forçar durante o comissionamento.

DESIGNER PRÁTICO

Copiar, colar, alinhar e distribuir widgets; um duplo clique abre as Propriedades.

COMPILAÇÃO INTEGRADA

Painel com os erros marcados no editor. MinGW ou Visual Studio no Windows, g++ no Linux, com o Qt de cada projeto escolhido na barra.

QUATRO IDIOMAS

pvdevelop e pvbrowser em espanhol, inglês, português e alemão, com manual e ajuda integrada.

Primeiros passos

1. No pvdevelop, **File > New**: pasta e nome do projeto.
2. **Daemon > Modbus**: endereço do CLP e a tabela de tags. Ao compilar, são gerados o daemon e seu `.tagmap` .
3. No Designer, insira os widgets e use **Tags...** nas Propriedades para dar-lhes ações e animações.
4. **Ctrl+T** compila e executa o servidor; **Ctrl+P** abre o pvbrowser para vê-lo.
5. **Daemon > Monitor de tags** para observar e forçar valores. O exemplo `01_modbus` traz tudo pronto.

Instalação

Windows 64 bits: um instalador que deixa tudo em `C:\PVSuite`. Se só for preciso observar, dá para instalar apenas o pvbrowser.

Linux x86-64: um `.run` que instala em `/opt/pvb` com o Qt incluído. Se faltar alguma biblioteca no desktop, o `preparar_linux.sh` a instala.

macOS (Apple Silicon): pvbrowser e pvdevelop num zip, ainda em beta e sem assinatura.

Se já estiver instalado, o instalador oferece a atualização: não é preciso apagar nada à mão.

LICENÇA

pvbrowser e pvdevelop são GPL. pvserver e rllib são LGPL, então os servidores que você montar podem ter a licença que quiser.

PROJETOS ANTERIORES

Os projetos do pvbrowser continuam funcionando. Com o componente MinGW 8.1 compilam como antes, em 32 ou 64 bits, e depois passam ao Qt 6 com calma.

Algumas palavras

O PV Suite nasce de usar o pvbrowser de verdade e de querer que o tedioso deixe de ser. Cada função veio de uma necessidade concreta: um botão que precisava escrever um registro, uma tela que não avisava que o daemon tinha caído.

É software livre e está em beta: dá para testar, quebrar e melhorar. Se algo não funcionar como você espera, conte no GitHub; é o melhor caminho para a próxima versão sair melhor.

Néstor Palomeque

ONDE ENCONTRAR

CÓDIGO-FONTE

`github.com/nestorlevi/PvSuite`

INSTALADORES

`github.com/nestorlevi/PvSuite/releases`

APOIAR O PROJETO

`cafecito.app/nestorlevi`

Se for útil para você, pode me pagar um café (Cafecito, Argentina).

BASEADO EM

`pvbrowser.de` — Rainer Lehrig